BỘ TÀI LIỆU 4: LỘ TRÌNH REACTJS - TRẢI NGHIỆM NGƯỜI DÙNG TÍCH HỢP AI

AI Experience Plane | React 19+ + TypeScript + Vite | Chuẩn OJT 2026 | Phiên bản v2

R2S Academy - OJT 2026

1. ĐỊNH VỊ TÀI LIỆU & CAM KẾT ĐẦU RA

Tài liệu này dành cho lập trình viên ReactJS, Fresher Frontend, Junior Frontend, học viên OJT 2026 và Mentor cần một bộ khung thực chiến để thiết kế giao diện AI-First. Đây không phải là tài liệu dạy React theo kiểu dựng form, gọi API rồi render bảng dữ liệu. Thị trường 2026 không còn thiếu người biết làm CRUD tĩnh. Thị trường thiếu người biết biến React Frontend thành **AI Experience Plane** - lớp trải nghiệm nơi người dùng nhìn thấy AI đang làm gì, hiểu vì sao hệ thống chờ, quyết định có tin đề xuất hay không và luôn có quyền dừng những hành động rủi ro.

Nghịch lý tuyển dụng năm 2026 rất thẳng: doanh nghiệp có thể dùng AI để sinh component UI, tự fix lỗi CSS cơ bản, generate form và viết test skeleton trong vài phút. Nhưng doanh nghiệp vẫn khát Frontend Engineer biết thiết kế luồng streaming, xử lý dữ liệu bất định, chống XSS từ AI output, chặn PII trước khi gửi lên Cloud AI, đảm bảo A11Y và giải trình được logic phòng thủ trước Tech Lead.

Nếu Java Backend là lớp **AI Control Plane** kiểm soát quyền thực thi, thì React Frontend là lớp **AI Experience Plane**: nơi niềm tin của người dùng được xây hoặc bị phá vỡ.

1.1. Cam kết đầu ra

Sau khi học xong tài liệu này, học viên phải tạo được ít nhất 6 đầu ra:

1. Một giao diện AI-First có loading state theo tiến trình thật.
2. Một luồng SSE hoặc Fetch Streaming để nhận dữ liệu AI theo thời gian thực.
3. Một Error Boundary bảo vệ các component có AI output không ổn định.
4. Một lớp sanitize output để chống XSS từ nội dung AI sinh ra.
5. Một lớp kiểm soát A11Y và PII ở giao diện trước khi gửi dữ liệu lên AI Cloud.
6. Một cơ chế hard-block dữ liệu Restricted trước khi người dùng submit.

1.2. Stack đề xuất cho OJT 2026

Thành phần	Khuyến nghị
Frontend	React 19+, TypeScript, Vite
State cục bộ	useState, useReducer, useRef, custom hooks
Server state	TanStack Query hoặc fetch thuần ở mức học tập
Realtime	Server-Sent Events, Fetch Streaming API
UX control	Error Boundary, AbortController, Progressive Loading
Security UI	Sanitize output, PII warning, Restricted data hard-block
A11Y	aria-live, role="status", role="alert", keyboard first
Delivery	Pull Request kèm checklist, negative test cases và demo evidence

Thông điệp chính: AI có thể sinh câu trả lời. Frontend Engineer giỏi phải thiết kế trải nghiệm để người dùng hiểu hệ thống đang làm gì, dừng được khi cần, xác nhận trước hành động nhạy cảm và không bị mắc kẹt trong spinner vô hạn.

2. CHƯƠNG 1: TƯ DUY GIAO DIỆN AI - AI-FIRST UI/UX THINKING

2.1. Problem Statement

Giao diện truyền thống được xây cho thao tác rõ ràng: mở form, nhập dữ liệu, nhấn submit, nhận kết quả. Nhưng sản phẩm AI không vận hành như vậy. AI thường cần thời gian đọc dữ liệu, phân tích, sinh kết quả, tự đánh giá rủi ro và đôi khi cần người dùng xác nhận lại trước khi thực thi.

Nếu Frontend Dev vẫn thiết kế theo kiểu CRUD tĩnh, người dùng sẽ gặp 5 vấn đề:

- Không biết AI đang xử lý bước nào.
- Không biết kết quả AI là đề xuất hay quyết định cuối cùng.
- Không có nút dừng khi AI đang sinh nội dung dài.
- Không có cảnh báo khi AI chuẩn bị thực hiện hành động nhạy cảm.
- Không biết dữ liệu cá nhân nào đang bị gửi lên hệ thống AI.

Trong năm 2026, UI không chỉ là nơi hiển thị dữ liệu. UI phải trở thành lớp điều phối trải nghiệm, giúp con người kiểm soát AI thay vì bị AI kéo đi.

2.2. Giải pháp tư duy

Từ giao diện tĩnh sang giao diện hội thoại

Conversational UI không chỉ là chatbot. Đó là kiểu giao diện cho phép người dùng trao đổi theo ngữ cảnh, nhận phản hồi từng bước và luôn biết AI đang xử lý phần nào. Một màn hình AI tốt phải trả lời được 3 câu hỏi: hệ thống đang làm gì, dữ liệu nào đang được dùng, và người dùng có thể can thiệp ở đâu.

Từ màn hình cố định sang Generative UI

Generative UI là giao diện có thể thay đổi theo output của AI: AI gợi ý action card, checklist, câu hỏi phỏng vấn, tóm tắt rủi ro hoặc form tiếp theo. Nhưng Generative UI không được phép biến thành "AI muốn vẽ gì thì vẽ". Frontend phải có type, schema guard và component whitelist.

Human-in-the-loop là nguyên tắc bắt buộc

AI được phép đề xuất. Người dùng hoặc Backend mới được quyền quyết định. Với hành động nhạy cảm như thanh toán, xóa dữ liệu, gửi email hàng loạt, thay đổi trạng thái hồ sơ, khóa tài khoản hoặc hoàn tiền, giao diện phải có bước xác nhận rõ ràng.

Optimistic UI phải có rollback

Optimistic UI giúp giao diện phản hồi nhanh trước khi server xử lý xong. Nhưng trong sản phẩm AI, optimistic UI phải luôn có rollback vì AI có thể timeout, trả về low confidence hoặc bị backend từ chối vì rủi ro chính sách.

2.3. Bảng tư duy mới cho ReactJS AI-First

Tiêu chí	Frontend CRUD truyền thống	AI-First Frontend Engineer chuẩn OJT 2026
Mục tiêu UI	Hiển thị form/bảng đúng dữ liệu	Điều phối trải nghiệm AI, trạng thái bất định và quyền kiểm soát của user
Output	Dữ liệu xác định từ API	Dữ liệu có thể streaming, thiếu field, sai format hoặc cần xác thực
Loading	Spinner chung chung	Progressive state theo pipeline thật
Action	Click là thực thi	AI đề xuất, user xác nhận, backend thực thi
Security UI	Validate form cơ bản	Sanitize AI output, chặn XSS, cảnh báo PII, hard-block Restricted data
A11Y	Có label là đủ	Live region, alert region, keyboard flow, thông báo không gây nhiễu screen reader
Review PR	UI giống design	UI có fallback, cancel, rollback, safe rendering, test evidence

2.4. Code mẫu: AI Action Card có Human Confirmation

src/features/ai-actions/types.ts

```ts
export type AiActionRiskLevel = 'LOW' | 'MEDIUM' | 'HIGH';

export type AiSuggestedAction = {
  id: string;
  title: string;
  description: string;
  riskLevel: AiActionRiskLevel;
  requiresConfirmation: boolean;
  actionLabel: string;
};

export type ConfirmedAiActionPayload = {
  actionId: string;
  confirmedAt: string;
  confirmedBy: string;
};
```

src/features/ai-actions/ConfirmActionDialog.tsx

```tsx
import { useEffect, useRef } from 'react';
import type { AiSuggestedAction } from './types';

type ConfirmActionDialogProps = {
  action: AiSuggestedAction;
  isOpen: boolean;
  onCancel: () => void;
  onConfirm: () => void;
};

export function ConfirmActionDialog({
  action,
  isOpen,
  onCancel,
  onConfirm,
}: ConfirmActionDialogProps) {
  const confirmButtonRef = useRef<HTMLButtonElement | null>(null);

  useEffect(() => {
    if (isOpen) {
      confirmButtonRef.current?.focus();
    }
  }, [isOpen]);

  if (!isOpen) {
    return null;
  }

  return (
```

```
    <div className="dialog-backdrop" role="presentation">
      <section
        className="confirm-dialog"
        role="dialog"
        aria-modal="true"
        aria-labelledby="confirm-dialog-title"
        aria-describedby="confirm-dialog-description"
      >
        <h2 id="confirm-dialog-title">Xác nhận hành động do AI đề xuất</h2>
        <p id="confirm-dialog-description">
          AI chỉ đang đề xuất. Bạn là người quyết định cuối cùng trước khi hệ thống thực thi.
        </p>

        <div className="confirm-dialog__summary">
          <strong>{action.title}</strong>
          <p>{action.description}</p>
          <span className={`risk-badge risk-badge--${action.riskLevel.toLowerCase()}`}>
            Risk: {action.riskLevel}
          </span>
        </div>

        <div className="confirm-dialog__actions">
          <button type="button" className="button button--secondary" onClick={onCancel}>
            Hủy
          </button>
          <button
            ref={confirmButtonRef}
            type="button"
            className="button button--danger"
            onClick={onConfirm}
          >
            Tôi hiểu rủi ro và xác nhận
          </button>
        </div>
      </section>
    </div>
  );
}
```

src/features/ai-actions/AiSuggestedActionCard.tsx

```
import { useState } from 'react';
import { ConfirmActionDialog } from './ConfirmActionDialog';
import type { AiSuggestedAction, ConfirmedAiActionPayload } from './types';

type AiSuggestedActionCardProps = {
  action: AiSuggestedAction;
  currentUserId: string;
  onExecute: (payload: ConfirmedAiActionPayload) => Promise<void>;
};

export function AiSuggestedActionCard({
  action,
  currentUserId,
  onExecute,
}: AiSuggestedActionCardProps) {
  const [isDialogOpen, setIsDialogOpen] = useState(false);
  const [isExecuting, setIsExecuting] = useState(false);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const executeConfirmedAction = async () => {
    setIsExecuting(true);
    setErrorMessage(null);

    try {
      await onExecute({
        actionId: action.id,
        confirmedAt: new Date().toISOString(),
        confirmedBy: currentUserId,
      });
      setIsDialogOpen(false);
    } catch (error) {
      const message = error instanceof Error ? error.message : 'Không thể thực thi hành động.';
      setErrorMessage(message);
    } finally {
      setIsExecuting(false);
    }
  };

  const handlePrimaryClick = () => {
    if (action.requiresConfirmation || action.riskLevel !== 'LOW') {
      setIsDialogOpen(true);
      return;
    }

    void executeConfirmedAction();
  };

  return (
    <article className="ai-action-card" aria-labelledby={`ai-action-${action.id}`}>
```

```
      <header className="ai-action-card__header">
        <span className="ai-chip">AI Suggested</span>
        <span className={`risk-badge risk-badge--${action.riskLevel.toLowerCase()}`}>
          {action.riskLevel}
        </span>
      </header>

      <h3 id={`ai-action-${action.id}`}>{action.title}</h3>
      <p>{action.description}</p>

      {errorMessage && (
        <p className="form-error" role="alert">
          {errorMessage}
        </p>
      )}

      <button
        type="button"
        className="button button--primary"
        disabled={isExecuting}
        onClick={handlePrimaryClick}
      >
        {isExecuting ? 'Đang xử lý...' : action.actionLabel}
      </button>

      <ConfirmActionDialog
        action={action}
        isOpen={isDialogOpen}
        onCancel={() => setIsDialogOpen(false)}
        onConfirm={executeConfirmedAction}
      />
    </article>
  );
}
```

2.5. Code mẫu: Optimistic UI có rollback

src/features/ai-actions/useOptimisticAiAction.ts

```
import { useCallback, useState } from 'react';

type OptimisticStatus = 'IDLE' | 'OPTIMISTIC_APPLIED' | 'CONFIRMED' | 'ROLLED_BACK';

type UseOptimisticAiActionOptions<TState, TPayload> = {
  initialState: TState;
  applyOptimistic: (currentState: TState, payload: TPayload) => TState;
  rollback: (previousState: TState) => TState;
  execute: (payload: TPayload) => Promise<TState>;
};

export function useOptimisticAiAction<TState, TPayload>({
```

```
  initialState,
  applyOptimistic,
  rollback,
  execute,
}: UseOptimisticAiActionOptions<TState, TPayload>) {
  const [state, setState] = useState<TState>(initialState);
  const [status, setStatus] = useState<OptimisticStatus>('IDLE');
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const run = useCallback(
    async (payload: TPayload) => {
      const previousState = state;
      const optimisticState = applyOptimistic(state, payload);

      setState(optimisticState);
      setStatus('OPTIMISTIC_APPLIED');
      setErrorMessage(null);

      try {
        const confirmedState = await execute(payload);
        setState(confirmedState);
        setStatus('CONFIRMED');
      } catch (error) {
        setState(rollback(previousState));
        setStatus('ROLLED_BACK');
        setErrorMessage(error instanceof Error ? error.message : 'AI action failed and was rolled
back.');
      }
    },
    [applyOptimistic, execute, rollback, state],
  );

  return {
    state,
    status,
    errorMessage,
    run,
  };
}
```

2.6. Sanitize Output chống XSS từ AI

AI có thể sinh markdown, HTML fragment, link, mô tả action hoặc câu trả lời dạng text dài. Nếu Frontend render thẳng output bằng dangerouslySetInnerHTML, một đoạn như <img src=x onerror=alert(document.cookie)> có thể biến thành XSS. Với sản phẩm AI-First, **AI output phải được xem là untrusted input**.

Nguyên tắc OJT 2026:

- Không dùng dangerouslySetInnerHTML cho output AI nếu không có sanitize.
- Không tin rằng “AI sẽ không sinh script”. AI có thể lặp lại payload độc hại từ user prompt.
- Ưu tiên render text thuần. Nếu bắt buộc hỗ trợ HTML subset, phải sanitize bằng allowlist.
- Không lưu output thô vào storage dài hạn ở frontend.

src/shared/utils/sanitizeHtml.ts

```
const ALLOWED_TAGS = new Set([
  'b',
  'strong',
  'i',
  'em',
  'u',
  'p',
  'br',
  'ul',
  'ol',
  'li',
  'code',
  'pre',
  'blockquote',
  'span',
]);

const ALLOWED_ATTRIBUTES = new Set(['class', 'title']);

function removeDangerousProtocol(value: string): string {
  const normalized = value.trim().toLowerCase().replace(/\s+/g, '');

  if (
    normalized.startsWith('javascript:') ||
    normalized.startsWith('data:text/html') ||
    normalized.startsWith('vbscript:')
  ) {
    return '#';
  }

  return value;
}

function sanitizeElement(element: Element): void {
  const tagName = element.tagName.toLowerCase();

  if (!ALLOWED_TAGS.has(tagName)) {
    const textNode = document.createTextNode(element.textContent ?? '');
    element.replaceWith(textNode);
    return;
```

```
  }

  for (const attribute of Array.from(element.attributes)) {
    const name = attribute.name.toLowerCase();
    const value = attribute.value;
    const isEventHandler = name.startsWith('on');
    const isDangerousStyle = name === 'style';
    const isSrcDoc = name === 'srcdoc';
    const isHrefOrSrc = name === 'href' || name === 'src';

    if (isEventHandler || isDangerousStyle || isSrcDoc) {
      element.removeAttribute(attribute.name);
      continue;
    }

    if (isHrefOrSrc) {
      element.setAttribute(attribute.name, removeDangerousProtocol(value));
      continue;
    }

    if (!ALLOWED_ATTRIBUTES.has(name)) {
      element.removeAttribute(attribute.name);
    }
  }
}

export function sanitizeHtml(rawHtml: string): string {
  const template = document.createElement('template');
  template.innerHTML = rawHtml;

  const scripts = template.content.querySelectorAll('script, iframe, object, embed, link, meta');
  scripts.forEach((node) => node.remove());

  const elements = template.content.querySelectorAll('*');
  elements.forEach((element) => sanitizeElement(element));

  return template.innerHTML;
}

export function stripAllHtml(rawValue: string): string {
  const template = document.createElement('template');
  template.innerHTML = rawValue;
  return template.content.textContent ?? '';
}
```

src/shared/components/SafeAiText.tsx

```
import { useMemo } from 'react';
import { sanitizeHtml } from '../utils/sanitizeHtml';

type SafeAiTextProps = {
  value: string;
  mode?: 'plain-text' | 'safe-html';
  className?: string;
};

export function SafeAiText({ value, mode = 'plain-text', className }: SafeAiTextProps) {
  const sanitizedHtml = useMemo(() => sanitizeHtml(value), [value]);

  if (mode === 'plain-text') {
    return <p className={className}>{value}</p>;
  }

  return <div className={className} dangerouslySetInnerHTML={{ __html: sanitizedHtml }} />;
}
```

src/features/ai-actions/AiSuggestedActionCard.tsx

```
import { useState } from 'react';
import { SafeAiText } from '../../shared/components/SafeAiText';
import { ConfirmActionDialog } from './ConfirmActionDialog';
import type { AiSuggestedAction, ConfirmedAiActionPayload } from './types';

type AiSuggestedActionCardProps = {
  action: AiSuggestedAction;
  currentUserId: string;
  onExecute: (payload: ConfirmedAiActionPayload) => Promise<void>;
};

export function AiSuggestedActionCard({
  action,
  currentUserId,
  onExecute,
}: AiSuggestedActionCardProps) {
  const [isDialogOpen, setIsDialogOpen] = useState(false);
  const [isExecuting, setIsExecuting] = useState(false);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const executeConfirmedAction = async () => {
    setIsExecuting(true);
    setErrorMessage(null);

    try {
      await onExecute({
```

```
        actionId: action.id,
        confirmedAt: new Date().toISOString(),
        confirmedBy: currentUserId,
      });
      setIsDialogOpen(false);
    } catch (error) {
      const message = error instanceof Error ? error.message : 'Không thể thực thi hành động.';
      setErrorMessage(message);
    } finally {
      setIsExecuting(false);
    }
  };

  const handlePrimaryClick = () => {
    if (action.requiresConfirmation || action.riskLevel !== 'LOW') {
      setIsDialogOpen(true);
      return;
    }

    void executeConfirmedAction();
  };

  return (
    <article className="ai-action-card" aria-labelledby={`ai-action-${action.id}`}>
      <header className="ai-action-card__header">
        <span className="ai-chip">AI Suggested</span>
        <span className={`risk-badge risk-badge--${action.riskLevel.toLowerCase()}`}>
          {action.riskLevel}
        </span>
      </header>

      <h3 id={`ai-action-${action.id}`}>{action.title}</h3>
      <SafeAiText value={action.description} mode="safe-html"
className="ai-action-card__description" />

      {errorMessage && (
        <p className="form-error" role="alert">
          {errorMessage}
        </p>
      )}

      <button
        type="button"
        className="button button--primary"
        disabled={isExecuting}
        onClick={handlePrimaryClick}
      >
        {isExecuting ? 'Đang xử lý...' : action.actionLabel}
      </button>
```

```
      <ConfirmActionDialog
        action={action}
        isOpen={isDialogOpen}
        onCancel={() => setIsDialogOpen(false)}
        onConfirm={executeConfirmedAction}
      />
    </article>
  );
}
```

2.7. Mini-checklist trước khi mở PR

Mục kiểm tra	**Đạt khi**
AI chỉ đề xuất	UI không tự động thực thi action nhạy cảm
Có confirmation	Action rủi ro MEDIUM/HIGH bắt buộc mở dialog xác nhận
Có rollback	Optimistic update có khả năng hoàn tác khi backend từ chối
Không render output thô	AI output được render text thuần hoặc sanitize trước khi dùng HTML
Không dùng storage bừa bãi	Không lưu action payload nhạy cảm vào localStorage

3. CHƯƠNG 2: QUẢN TRỊ TRẠNG THÁI BẤT ĐỊNH VÀ ĐỘ TRỄ

3.1. Problem Statement

AI có độ trễ lớn hơn API CRUD. Một request tóm tắt tài liệu, phân tích mã nguồn hoặc review CV có thể mất 5-30 giây. Nếu UI chỉ hiển thị spinner vô hạn, người dùng sẽ nghĩ hệ thống treo. Nếu AI trả về sai format, component có thể crash. Nếu toàn bộ app bị sập chỉ vì một panel AI lỗi, đó là thiết kế frontend yếu.

Frontend AI-First phải quản trị 3 loại bất định:

1. Bất định thời gian: không biết AI hoàn tất khi nào.
2. Bất định dữ liệu: output có thể thiếu field hoặc sai schema.
3. Bất định độ tin cậy: output có thể low confidence hoặc hallucination.

3.2. Giải pháp tư duy

Loading state phải có tiến trình thật

Thay vì "Loading…", UI cần hiển thị pipeline: RECEIVED -> READING_FILE -> ANALYZING -> SECURITY_CHECKING -> FINALIZING.

Error Boundary phải bọc vùng rủi ro cao

Các component hiển thị AI output phải được bọc bởi Error Boundary. Khi AI output sai format hoặc component parse lỗi, fallback UI phải xuất hiện trong vùng đó, không làm sập toàn bộ trang.

Schema guard ở Frontend không thay thế Backend

Frontend schema guard giúp tránh crash UI và hiển thị fallback sớm. Nhưng quyền quyết định dữ liệu hợp lệ vẫn thuộc về Backend. Không dùng Frontend guard để thay thế validation server-side.

3.3. Code mẫu: Loading state theo tiến trình AI

src/features/ai-progress/aiProgressTypes.ts

```ts
export type AiProgressStepId =
  | 'RECEIVED'
  | 'READING_FILE'
  | 'ANALYZING'
  | 'SECURITY_CHECKING'
  | 'FINALIZING'
  | 'COMPLETED'
  | 'FAILED';

export type AiProgressStep = {
  id: AiProgressStepId;
  label: string;
  description: string;
};
```

```ts
export const AI_PROGRESS_STEPS: AiProgressStep[] = [
  {
    id: 'RECEIVED',
    label: 'Đã nhận yêu cầu',
    description: 'Hệ thống đã nhận dữ liệu và đưa vào hàng đợi xử lý.',
  },
  {
    id: 'READING_FILE',
    label: 'Đang đọc dữ liệu',
    description: 'AI đang đọc nội dung đầu vào và chuẩn hóa ngữ cảnh.',
  },
  {
    id: 'ANALYZING',
    label: 'Đang phân tích bằng AI',
    description: 'Mô hình đang bóc tách ý chính, rủi ro và đề xuất.',
  },
  {
    id: 'SECURITY_CHECKING',
    label: 'Đang kiểm tra bảo mật',
    description: 'Hệ thống rà soát PII, prompt injection và output schema.',
  },
  {
    id: 'FINALIZING',
    label: 'Đang hoàn tất kết quả',
    description: 'Backend đang kiểm tra policy và chuẩn bị phản hồi cuối cùng.',
  },
  {
    id: 'COMPLETED',
    label: 'Hoàn tất',
    description: 'Kết quả đã sẵn sàng để người dùng xem xét.',
  },
  {
    id: 'FAILED',
    label: 'Không thể hoàn tất',
    description: 'Yêu cầu đã được chuyển sang fallback an toàn.',
  },
];
```

src/features/ai-progress/AiProgressTimeline.tsx

```ts
import type { AiProgressStepId } from './aiProgressTypes';
import { AI_PROGRESS_STEPS } from './aiProgressTypes';

type AiProgressTimelineProps = {
  currentStep: AiProgressStepId;
  failedReason?: string;
};

export function AiProgressTimeline({ currentStep, failedReason }: AiProgressTimelineProps) {
  const currentIndex = AI_PROGRESS_STEPS.findIndex((step) => step.id === currentStep);
```

```
  return (
    <ol className="ai-progress" aria-label="Tiến trình xử lý AI">
      {AI_PROGRESS_STEPS.filter((step) => step.id !== 'FAILED' || currentStep ===
'FAILED').map(
        (step, index) => {
          const isActive = step.id === currentStep;
          const isDone = currentIndex >= 0 && index < currentIndex;
          const isFailed = step.id === 'FAILED';

          return (
            <li
              key={step.id}
              className={[
                'ai-progress__item',
                isActive ? 'ai-progress__item--active' : '',
                isDone ? 'ai-progress__item--done' : '',
                isFailed ? 'ai-progress__item--failed' : '',
              ].join(' ')}
              aria-current={isActive ? 'step' : undefined}
            >
              <span className="ai-progress__dot" aria-hidden="true" />
              <div>
                <strong>{step.label}</strong>
                <p>{isFailed && failedReason ? failedReason : step.description}</p>
              </div>
            </li>
          );
        },
      )}
    </ol>
  );
}
```

3.4. Code mẫu: Error Boundary cho AI component

src/shared/error/AiErrorBoundary.tsx

```
import { Component, type ErrorInfo, type ReactNode } from 'react';

type AiErrorBoundaryProps = {
  children: ReactNode;
  fallbackTitle?: string;
  fallbackMessage?: string;
  onError?: (error: Error, errorInfo: ErrorInfo) => void;
};

type AiErrorBoundaryState = {
  hasError: boolean;
  errorMessage: string | null;
};

export class AiErrorBoundary extends Component<AiErrorBoundaryProps,
```

```
AiErrorBoundaryState> {
  state: AiErrorBoundaryState = {
    hasError: false,
    errorMessage: null,
  };

  static getDerivedStateFromError(error: Error): AiErrorBoundaryState {
    return {
      hasError: true,
      errorMessage: error.message,
    };
  }

  componentDidCatch(error: Error, errorInfo: ErrorInfo) {
    this.props.onError?.(error, errorInfo);
  }

  reset = () => {
    this.setState({
      hasError: false,
      errorMessage: null,
    });
  };

  render() {
    if (!this.state.hasError) {
      return this.props.children;
    }

    return (
      <section className="ai-fallback" role="alert" aria-live="assertive">
        <h2>{this.props.fallbackTitle ?? 'Không thể hiển thị kết quả AI'}</h2>
        <p>
          {this.props.fallbackMessage ??
            'Kết quả AI không đạt định dạng an toàn. Hệ thống đã chặn hiển thị để tránh gây hiểu
nhầm.'}
        </p>
        {this.state.errorMessage && <small>Mã lỗi kỹ thuật: {this.state.errorMessage}</small>}
        <button type="button" className="button button--secondary" onClick={this.reset}>
          Thử hiển thị lại
        </button>
      </section>
    );
  }
}
```

src/features/ai-result/AiResultSchemaGuard.tsx

```
import type { ReactNode } from 'react';

export type AiSummaryResult = {
  summary: string;
  confidenceScore: number;
  riskFlags: string[];
  suggestedNextActions: string[];
};

function isStringArray(value: unknown): value is string[] {
  return Array.isArray(value) && value.every((item) => typeof item === 'string');
}

export function parseAiSummaryResult(value: unknown): AiSummaryResult {
  if (typeof value !== 'object' || value === null) {
    throw new Error('AI_RESULT_NOT_OBJECT');
  }

  const record = value as Record<string, unknown>;

  if (typeof record.summary !== 'string' || record.summary.trim().length === 0) {
    throw new Error('AI_RESULT_INVALID_SUMMARY');
  }

  if (typeof record.confidenceScore !== 'number' || record.confidenceScore < 0 ||
record.confidenceScore > 1) {
    throw new Error('AI_RESULT_INVALID_CONFIDENCE');
  }

  if (!isStringArray(record.riskFlags)) {
    throw new Error('AI_RESULT_INVALID_RISK_FLAGS');
  }

  if (!isStringArray(record.suggestedNextActions)) {
    throw new Error('AI_RESULT_INVALID_ACTIONS');
  }

  return {
    summary: record.summary,
    confidenceScore: record.confidenceScore,
    riskFlags: record.riskFlags,
    suggestedNextActions: record.suggestedNextActions,
  };
}
```

```
type AiResultSchemaGuardProps = {
  value: unknown;
  children: (result: AiSummaryResult) => ReactNode;
};

export function AiResultSchemaGuard({ value, children }: AiResultSchemaGuardProps) {
  const result = parseAiSummaryResult(value);
  return <>{children(result)}</>;
}
```

src/features/ai-result/AiSummaryPanel.tsx

```
import { AiErrorBoundary } from '../../shared/error/AiErrorBoundary';
import { AiResultSchemaGuard } from './AiResultSchemaGuard';

type AiSummaryPanelProps = {
  rawResult: unknown;
};

export function AiSummaryPanel({ rawResult }: AiSummaryPanelProps) {
  return (
    <AiErrorBoundary
      fallbackTitle="Kết quả AI chưa đủ an toàn để hiển thị"
      fallbackMessage="Frontend đã chặn output vì dữ liệu không đúng schema hoặc thiếu
trường bắt buộc."
    >
      <AiResultSchemaGuard value={rawResult}>
        {(result) => (
          <section className="ai-summary-panel" aria-labelledby="ai-summary-title">
            <h2 id="ai-summary-title">Tóm tắt từ AI</h2>
            <p>{result.summary}</p>
            <p>
              <strong>Confidence:</strong> {(result.confidenceScore * 100).toFixed(0)}%
            </p>

            {result.riskFlags.length > 0 && (
              <div className="risk-list" role="status">
                <strong>Risk flags:</strong>
                <ul>
                  {result.riskFlags.map((flag) => (
                    <li key={flag}>{flag}</li>
                  ))}
                </ul>
              </div>
            )}

            <h3>Hành động gợi ý</h3>
            <ul>
              {result.suggestedNextActions.map((action) => (
                <li key={action}>{action}</li>
              ))}
```

```
          </ul>
        </section>
      )}
    </AiResultSchemaGuard>
  </AiErrorBoundary>
 );
}
```

3.5. Mini-checklist trước khi mở PR

Mục kiểm tra	Đạt khi
Loading có nghĩa	Người dùng biết AI đang ở bước nào
Không crash toàn app	Component AI lỗi chỉ fallback trong vùng rủi ro
Có schema guard	Output thiếu field bị chặn trước khi render
Fallback trung tính	Không đổ lỗi, không lộ rule nội bộ
Có test dữ liệu lỗi	QA có case output sai format, timeout, low confidence

4. CHƯƠNG 3: TÍCH HỢP LUỒNG DỮ LIỆU THỜI GIAN THỰC

4.1. Problem Statement

AI không luôn trả kết quả một lần. Có khi backend phát trạng thái task theo SSE. Có khi model stream token từng phần. Nếu Frontend không biết xử lý stream, app sẽ rơi vào 3 lỗi phổ biến: treo UI, re-render quá nhiều gây lag, hoặc không hủy được request khi người dùng muốn dừng.

4.2. Giải pháp tư duy

Khi nào dùng SSE?

Dùng SSE khi backend muốn đẩy trạng thái một chiều về client: task received, sanitizing, calling AI, validating, completed, failed. SSE đơn giản hơn WebSocket cho dashboard trạng thái, log tiến trình và thông báo một chiều.

Khi nào dùng Fetch Streaming?

Dùng Fetch Streaming khi endpoint trả text/token theo luồng và cần dùng AbortController để hủy request. Đây là cách phù hợp cho màn hình AI viết câu trả lời dài hoặc phân tích tài liệu.

Tối ưu render

Không update React state sau từng ký tự nếu output dài. Nên buffer text bằng useRef, flush theo nhịp 50-100ms hoặc theo chunk đủ lớn để tránh re-render component cha không cần thiết.

4.3. Code mẫu: Hook SSE nhận trạng thái task AI

src/features/ai-streaming/aiTaskTypes.ts

```ts
export type AiTaskStatus =
  | 'RECEIVED'
  | 'SANITIZING_INPUT'
  | 'CALLING_AI'
  | 'VALIDATING_OUTPUT'
  | 'CHECKING_POLICY'
  | 'COMPLETED'
  | 'FAILED';

export type AiTaskEvent = {
  taskId: string;
  status: AiTaskStatus;
  message: string;
  occurredAt: string;
};
```

src/features/ai-streaming/useAiTaskEvents.ts

```ts
import { useEffect, useState } from 'react';
import type { AiTaskEvent } from './aiTaskTypes';

type UseAiTaskEventsResult = {
  events: AiTaskEvent[];
  latestEvent: AiTaskEvent | null;
  isConnected: boolean;
  errorMessage: string | null;
};

export function useAiTaskEvents(taskId: string | null): UseAiTaskEventsResult {
  const [events, setEvents] = useState<AiTaskEvent[]>([]);
  const [isConnected, setIsConnected] = useState(false);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  useEffect(() => {
    if (!taskId) {
      setEvents([]);
      setIsConnected(false);
      setErrorMessage(null);
      return undefined;
    }

    const eventSource = new EventSource(`/api/ai/tasks/${taskId}/events`);

    eventSource.onopen = () => {
      setIsConnected(true);
      setErrorMessage(null);
    };
```

```
    eventSource.onmessage = (event) => {
      try {
        const parsed = JSON.parse(event.data) as AiTaskEvent;
        setEvents((current) => [...current, parsed]);
      } catch {
        setErrorMessage('Không thể đọc trạng thái AI từ server.');
      }
    };

    eventSource.onerror = () => {
      setIsConnected(false);
      setErrorMessage('Kết nối SSE bị gián đoạn. Hệ thống sẽ thử kết nối lại nếu server cho
phép.');
    };

    return () => {
      eventSource.close();
    };
  }, [taskId]);

  return {
    events,
    latestEvent: events.at(-1) ?? null,
    isConnected,
    errorMessage,
  };
}
```

src/features/ai-streaming/AiTaskStatusPanel.tsx

```
import { useAiTaskEvents } from './useAiTaskEvents';

type AiTaskStatusPanelProps = {
  taskId: string | null;
};

export function AiTaskStatusPanel({ taskId }: AiTaskStatusPanelProps) {
  const { events, latestEvent, isConnected, errorMessage } = useAiTaskEvents(taskId);

  if (!taskId) {
    return <p>Chưa có tác vụ AI nào được khởi tạo.</p>;
  }

  return (
    <section className="ai-task-status" aria-labelledby="ai-task-status-title">
      <h2 id="ai-task-status-title">Trạng thái xử lý AI</h2>
      <p role="status" aria-live="polite">
        {latestEvent ? latestEvent.message : 'Đang mở luồng theo dõi tiến trình...'}
      </p>
      <p className={isConnected ? 'text-success' : 'text-muted'}>
        {isConnected ? 'SSE connected' : 'SSE disconnected'}
```

```
      </p>

      {errorMessage && (
        <p className="form-error" role="alert">
          {errorMessage}
        </p>
      )}

      <ol className="event-list">
        {events.map((event) => (
          <li key={`${event.taskId}-${event.status}-${event.occurredAt}`}>
            <strong>{event.status}</strong> - {event.message}
          </li>
        ))}
      </ol>
    </section>
  );
}
```

4.4. Code mẫu: Fetch Streaming + AbortController

src/features/ai-streaming/useTextStream.ts

```
import { useCallback, useRef, useState } from 'react';

type StreamState = 'IDLE' | 'STREAMING' | 'COMPLETED' | 'ABORTED' | 'FAILED';

type UseTextStreamResult = {
  text: string;
  state: StreamState;
  errorMessage: string | null;
  start: (prompt: string) => Promise<void>;
  stop: () => void;
  reset: () => void;
};

export function useTextStream(endpoint: string): UseTextStreamResult {
  const [text, setText] = useState('');
  const [state, setState] = useState<StreamState>('IDLE');
  const [errorMessage, setErrorMessage] = useState<string | null>(null);
  const abortControllerRef = useRef<AbortController | null>(null);

  const stop = useCallback(() => {
    abortControllerRef.current?.abort();
    abortControllerRef.current = null;
    setState('ABORTED');
  }, []);

  const reset = useCallback(() => {
    abortControllerRef.current?.abort();
    abortControllerRef.current = null;
```

```
    setText('');
    setErrorMessage(null);
    setState('IDLE');
  }, []);

  const start = useCallback(
    async (prompt: string) => {
      abortControllerRef.current?.abort();

      const controller = new AbortController();
      abortControllerRef.current = controller;

      setText('');
      setState('STREAMING');
      setErrorMessage(null);

      try {
        const response = await fetch(endpoint, {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
          },
          body: JSON.stringify({ prompt }),
          signal: controller.signal,
        });

        if (!response.ok || !response.body) {
          throw new Error(`STREAM_REQUEST_FAILED_${response.status}`);
        }

        const reader = response.body.getReader();
        const decoder = new TextDecoder('utf-8');

        while (true) {
          const { value, done } = await reader.read();

          if (done) {
            break;
          }

          const chunk = decoder.decode(value, { stream: true });
          setText((current) => current + chunk);
        }

        setState('COMPLETED');
      } catch (error) {
        if (error instanceof DOMException && error.name === 'AbortError') {
          setState('ABORTED');
          return;
        }
```

```
        setState('FAILED');
        setErrorMessage(error instanceof Error ? error.message : 'Không thể stream kết quả AI.');
      } finally {
        abortControllerRef.current = null;
      }
    },
    [endpoint],
  );

  return {
    text,
    state,
    errorMessage,
    start,
    stop,
    reset,
  };
}
```

src/features/ai-streaming/StreamingAnswerBox.tsx

```
import type { ReactNode } from 'react';

type StreamingAnswerBoxProps = {
  text: string;
  state: 'IDLE' | 'STREAMING' | 'COMPLETED' | 'ABORTED' | 'FAILED';
  errorMessage: string | null;
  onStop: () => void;
  onReset: () => void;
};

function renderStateLabel(state: StreamingAnswerBoxProps['state']): ReactNode {
  const labels: Record<StreamingAnswerBoxProps['state'], string> = {
    IDLE: 'Chưa bắt đầu',
    STREAMING: 'AI đang sinh câu trả lời',
    COMPLETED: 'Hoàn tất',
    ABORTED: 'Đã dừng bởi người dùng',
    FAILED: 'Không thể hoàn tất',
  };

  return labels[state];
}

export function StreamingAnswerBox({
  text,
  state,
  errorMessage,
  onStop,
  onReset,
}: StreamingAnswerBoxProps) {
```

```
  const canStop = state === 'STREAMING';

  return (
    <section className="streaming-answer" aria-labelledby="streaming-answer-title">
      <header className="streaming-answer__header">
        <h2 id="streaming-answer-title">Kết quả AI Streaming</h2>
        <span role="status" aria-live="polite">
          {renderStateLabel(state)}
        </span>
      </header>

      {errorMessage && (
        <p className="form-error" role="alert">
          {errorMessage}
        </p>
      )}

      <div className="streaming-answer__content" aria-live="polite">
        {text || 'Kết quả sẽ xuất hiện tại đây...'}
      </div>

      <div className="button-row">
        <button type="button" className="button button--secondary" disabled={!canStop}
onClick={onStop}>
          Dừng sinh câu trả lời
        </button>
        <button type="button" className="button button--ghost" onClick={onReset}>
          Xóa kết quả tạm
        </button>
      </div>
    </section>
  );
}
```

src/features/ai-streaming/AiStreamingDemo.tsx

```
import { useState } from 'react';
import { StreamingAnswerBox } from './StreamingAnswerBox';
import { useTextStream } from './useTextStream';

export function AiStreamingDemo() {
  const [prompt, setPrompt] = useState('');
  const { text, state, errorMessage, start, stop, reset } = useTextStream('/api/ai/stream-answer');

  const handleSubmit = (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();

    if (prompt.trim().length === 0) {
      return;
    }
```

```
    void start(prompt.trim());
  };

  return (
    <main className="ai-streaming-demo">
      <form className="ai-prompt-form" onSubmit={handleSubmit}>
        <label htmlFor="ai-prompt">Nhập yêu cầu cho AI</label>
        <textarea
          id="ai-prompt"
          value={prompt}
          rows={5}
          maxLength={5000}
          onChange={(event) => setPrompt(event.target.value)}
        />
        <button type="submit" className="button button--primary" disabled={state ===
'STREAMING'}>
          Bắt đầu stream
        </button>
      </form>

      <StreamingAnswerBox
        text={text}
        state={state}
        errorMessage={errorMessage}
        onStop={stop}
        onReset={reset}
      />
    </main>
  );
}
```

4.5. Tối ưu render word-by-word

src/features/ai-streaming/useBufferedTextStream.ts

```
import { useCallback, useRef, useState } from 'react';

type BufferedStreamState = 'IDLE' | 'STREAMING' | 'COMPLETED' | 'ABORTED' | 'FAILED';

type UseBufferedTextStreamResult = {
  text: string;
  state: BufferedStreamState;
  errorMessage: string | null;
  start: (prompt: string) => Promise<void>;
  stop: () => void;
};

export function useBufferedTextStream(endpoint: string): UseBufferedTextStreamResult {
  const [text, setText] = useState('');
  const [state, setState] = useState<BufferedStreamState>('IDLE');
  const [errorMessage, setErrorMessage] = useState<string | null>(null);
```

```
const abortControllerRef = useRef<AbortController | null>(null);
const bufferRef = useRef('');
const flushTimerRef = useRef<number | null>(null);

const flushBuffer = useCallback(() => {
  if (bufferRef.current.length === 0) {
    return;
  }

  const nextChunk = bufferRef.current;
  bufferRef.current = '';
  setText((current) => current + nextChunk);
}, []);

const scheduleFlush = useCallback(() => {
  if (flushTimerRef.current !== null) {
    return;
  }

  flushTimerRef.current = window.setTimeout(() => {
    flushTimerRef.current = null;
    flushBuffer();
  }, 80);
}, [flushBuffer]);

const stop = useCallback(() => {
  abortControllerRef.current?.abort();
  abortControllerRef.current = null;
  flushBuffer();
  setState('ABORTED');
}, [flushBuffer]);

const start = useCallback(
  async (prompt: string) => {
    abortControllerRef.current?.abort();

    const controller = new AbortController();
    abortControllerRef.current = controller;
    bufferRef.current = '';
    setText('');
    setErrorMessage(null);
    setState('STREAMING');

    try {
      const response = await fetch(endpoint, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({ prompt }),
```

```
        signal: controller.signal,
      });

      if (!response.ok || !response.body) {
        throw new Error(`STREAM_REQUEST_FAILED_${response.status}`);
      }

      const reader = response.body.getReader();
      const decoder = new TextDecoder('utf-8');

      while (true) {
        const { done, value } = await reader.read();

        if (done) {
          break;
        }

        bufferRef.current += decoder.decode(value, { stream: true });
        scheduleFlush();
      }

      flushBuffer();
      setState('COMPLETED');
    } catch (error) {
      if (error instanceof DOMException && error.name === 'AbortError') {
        flushBuffer();
        setState('ABORTED');
        return;
      }

      setState('FAILED');
      setErrorMessage(error instanceof Error ? error.message : 'Không thể stream dữ liệu AI.');
    } finally {
      abortControllerRef.current = null;

      if (flushTimerRef.current !== null) {
        window.clearTimeout(flushTimerRef.current);
        flushTimerRef.current = null;
      }
    }
  },
  [endpoint, flushBuffer, scheduleFlush],
);

return {
  text,
  state,
  errorMessage,
  start,
  stop,
```

```
  };
}
```

4.6. Không lưu stream thô hoặc prompt nhạy cảm vào Browser Storage

> **Cảnh báo kỹ thuật bắt buộc:** Tuyệt đối không lưu dữ liệu stream thô, prompt nhạy cảm, nội dung hội thoại AI, PII hoặc token phiên làm việc vào localStorage hoặc sessionStorage. Hai vùng storage này có thể bị đọc bởi JavaScript chạy trong cùng origin. Nếu ứng dụng dính XSS, attacker có thể đánh cắp toàn bộ nội dung lưu ở đó. Với AI-First UI, một đoạn prompt có thể chứa CV, email khách hàng, lịch sử khiếu nại, tài liệu nội bộ hoặc dữ liệu restricted. Lưu nó vào storage là biến trình duyệt thành kho rò rỉ dữ liệu.

Nguyên tắc xử lý:

- Prompt và stream tạm thời chỉ nằm trong RAM state hoặc useRef buffer.
- Khi component unmount, cleanup controller, buffer và timer.
- Nếu cần lưu lịch sử hội thoại, hãy lưu ở Backend đã phân quyền, mã hóa, có retention policy và audit log.
- Không dùng persist cho store chứa AI prompt/output nhạy cảm.

src/features/ai-streaming/useEphemeralAiBuffer.ts

```ts
import { useCallback, useEffect, useRef, useState } from 'react';

type UseEphemeralAiBufferResult = {
  visibleText: string;
  appendChunk: (chunk: string) => void;
  clear: () => void;
  snapshot: () => string;
};

export function useEphemeralAiBuffer(flushIntervalMs = 80): UseEphemeralAiBufferResult {
  const bufferRef = useRef('');
  const [visibleText, setVisibleText] = useState('');
  const timerRef = useRef<number | null>(null);

  const flush = useCallback(() => {
    if (bufferRef.current.length === 0) {
      return;
    }

    const chunk = bufferRef.current;
    bufferRef.current = '';
    setVisibleText((current) => current + chunk);
  }, []);

  const appendChunk = useCallback(
    (chunk: string) => {
      bufferRef.current += chunk;
```

```
      if (timerRef.current === null) {
        timerRef.current = window.setTimeout(() => {
          timerRef.current = null;
          flush();
        }, flushIntervalMs);
      }
    },
    [flush, flushIntervalMs],
  );

  const clear = useCallback(() => {
    bufferRef.current = '';
    setVisibleText('');

    if (timerRef.current !== null) {
      window.clearTimeout(timerRef.current);
      timerRef.current = null;
    }
  }, []);

  const snapshot = useCallback(() => {
    return visibleText + bufferRef.current;
  }, [visibleText]);

  useEffect(() => {
    return () => {
      bufferRef.current = '';

      if (timerRef.current !== null) {
        window.clearTimeout(timerRef.current);
        timerRef.current = null;
      }
    };
  }, []);

  return {
    visibleText,
    appendChunk,
    clear,
    snapshot,
  };
}
```

4.7. Mini-checklist trước khi mở PR

Mục kiểm tra	Đạt khi
SSE cleanup	EventSource.close() được gọi khi unmount
Abort stream	Người dùng có thể dừng AI đang sinh output
Tối ưu render	Có buffer, không setState từng ký tự trong output dài
Không lưu prompt nhạy cảm	Không ghi prompt/output AI vào localStorage/sessionStorage
Dữ liệu tạm được xóa	Buffer RAM được clear khi reset/unmount

5. CHƯƠNG 4: TÍNH TIẾP CẬN VÀ TRẢI NGHIỆM KHÔNG ĐỨT GÃY

5.1. Problem Statement

AI output thay đổi liên tục. Nếu không thiết kế A11Y đúng, người dùng dùng screen reader có thể bị đọc quá nhiều, đọc thiếu hoặc không biết hệ thống đã lỗi. Đồng thời, UI là chốt chặn đầu tiên trước khi dữ liệu cá nhân bị gửi lên AI Cloud. Nếu Frontend không cảnh báo hoặc không chặn dữ liệu nhạy cảm, người dùng có thể vô tình gửi mật khẩu, token, số thẻ hoặc thông tin khách hàng vào prompt.

5.2. Giải pháp tư duy

aria-live cho nội dung thay đổi

Dùng aria-live="polite" cho cập nhật tiến trình không khẩn cấp. Dùng role="alert" hoặc aria-live="assertive" cho lỗi cần người dùng biết ngay. Không để screen reader đọc lại toàn bộ câu trả lời AI sau mỗi token.

PII warning trước khi submit

Trước khi gửi prompt lên AI, UI cần quét nội dung để phát hiện email, số điện thoại, secret hoặc dữ liệu nhạy cảm. Với dữ liệu Confidential, có thể cho phép masking. Với dữ liệu Restricted, phải chặn đứng submit.

Fallback message phải trung tính

Không ghi “AI bị lỗi vì prompt injection rule X”. Hãy ghi “Yêu cầu cần được kiểm tra thủ công để đảm bảo an toàn”. UI không được lộ rule nội bộ cho attacker.

5.3. Code mẫu: PII detection ở UI

src/shared/privacy/piiDetection.ts

```ts
export type PiiType = 'EMAIL' | 'VIETNAM_PHONE' | 'SECRET' | 'PASSWORD' | 'PIN' |
'CREDIT_CARD';

export type DataRiskLevel = 'NONE' | 'PUBLIC' | 'INTERNAL' | 'CONFIDENTIAL' |
'RESTRICTED';

export type PiiDetection = {
  type: PiiType;
  value: string;
  startIndex: number;
  endIndex: number;
  riskLevel: DataRiskLevel;
};

export type DataClassificationResult = {
  riskLevel: DataRiskLevel;
  detections: PiiDetection[];
  canSubmit: boolean;
  requiresMasking: boolean;
  warningMessage: string | null;
};

const EMAIL_REGEX = /[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}/gi;
const VIETNAM_PHONE_REGEX = /(?:\+84|0)(?:3|5|7|8|9)\d{8}\b/g;
const SECRET_REGEX =
/\b(?:api[_-]?key|secret|token|access[_-]?token|refresh[_-]?token)\s*[:=]\s*["']?[A-Za-z0-9._\-]{12,
}["']?/gi;
const PASSWORD_REGEX = /\b(?:password|pwd|mat\s*khau|mật\s*khẩu)\s*[:=]\s*\S{6,}/gi;
const PIN_REGEX = /\b(?:pin|ma\s*pin|mã\s*pin)\s*[:=]\s*\d{4,8}\b/gi;
const CREDIT_CARD_REGEX = /\b(?:\d[ -]*?){13,19}\b/g;

function isLikelyCreditCard(value: string): boolean {
  const digits = value.replace(/\D/g, '');

  if (digits.length < 13 || digits.length > 19) {
    return false;
  }

  let sum = 0;
  let shouldDouble = false;

  for (let index = digits.length - 1; index >= 0; index -= 1) {
    let digit = Number(digits[index]);

    if (shouldDouble) {
      digit *= 2;
```

```
      if (digit > 9) {
        digit -= 9;
      }
    }

    sum += digit;
    shouldDouble = !shouldDouble;
  }

  return sum % 10 === 0;
}

function collectMatches(input: string, regex: RegExp, type: PiiType, riskLevel: DataRiskLevel):
PiiDetection[] {
  return Array.from(input.matchAll(regex)).map((match) => ({
    type,
    value: match[0],
    startIndex: match.index ?? 0,
    endIndex: (match.index ?? 0) + match[0].length,
    riskLevel,
  }));
}

export function detectPii(input: string): PiiDetection[] {
  const detections: PiiDetection[] = [
    ...collectMatches(input, EMAIL_REGEX, 'EMAIL', 'CONFIDENTIAL'),
    ...collectMatches(input, VIETNAM_PHONE_REGEX, 'VIETNAM_PHONE',
'CONFIDENTIAL'),
    ...collectMatches(input, SECRET_REGEX, 'SECRET', 'RESTRICTED'),
    ...collectMatches(input, PASSWORD_REGEX, 'PASSWORD', 'RESTRICTED'),
    ...collectMatches(input, PIN_REGEX, 'PIN', 'RESTRICTED'),
  ];

  for (const match of input.matchAll(CREDIT_CARD_REGEX)) {
    const value = match[0];

    if (isLikelyCreditCard(value)) {
      detections.push({
        type: 'CREDIT_CARD',
        value,
        startIndex: match.index ?? 0,
        endIndex: (match.index ?? 0) + value.length,
        riskLevel: 'RESTRICTED',
      });
    }
  }

  return detections.sort((left, right) => left.startIndex - right.startIndex);
}
```

```ts
export function maskDetectedPii(input: string, detections: PiiDetection[]): string {
  if (detections.length === 0) {
    return input;
  }

  let result = '';
  let cursor = 0;

  for (const detection of detections) {
    result += input.slice(cursor, detection.startIndex);
    result += `[${detection.type}_MASKED]`;
    cursor = detection.endIndex;
  }

  result += input.slice(cursor);
  return result;
}

export function classifyPromptData(input: string): DataClassificationResult {
  const detections = detectPii(input);
  const hasRestricted = detections.some((item) => item.riskLevel === 'RESTRICTED');
  const hasConfidential = detections.some((item) => item.riskLevel === 'CONFIDENTIAL');

  if (hasRestricted) {
    return {
      riskLevel: 'RESTRICTED',
      detections,
      canSubmit: false,
      requiresMasking: false,
      warningMessage:
        'Nội dung chứa dữ liệu Restricted như mật khẩu, PIN, token, secret hoặc số thẻ. Giao diện đã chặn gửi lên AI Cloud.',
    };
  }

  if (hasConfidential) {
    return {
      riskLevel: 'CONFIDENTIAL',
      detections,
      canSubmit: true,
      requiresMasking: true,
      warningMessage: 'Nội dung có dữ liệu cá nhân. Bạn phải masking trước khi gửi lên AI.',
    };
  }

  return {
    riskLevel: input.trim().length > 0 ? 'PUBLIC' : 'NONE',
    detections,
    canSubmit: input.trim().length > 0,
    requiresMasking: false,
```

```
    warningMessage: null,
  };
}
```

5.3.1. Code mẫu: Prompt Box an toàn bản

src/features/ai-input/SafeAiPromptBox.tsx

```
import { useMemo, useState } from 'react';
import { detectPii, maskDetectedPii } from '../../shared/privacy/piiDetection';

type SafeAiPromptBoxProps = {
  onSubmitPrompt: (prompt: string) => Promise<void>;
};

export function SafeAiPromptBox({ onSubmitPrompt }: SafeAiPromptBoxProps) {
  const [prompt, setPrompt] = useState('');
  const [shouldMaskPii, setShouldMaskPii] = useState(true);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const detections = useMemo(() => detectPii(prompt), [prompt]);
  const hasPii = detections.length > 0;
  const canSubmit = prompt.trim().length > 0 && !isSubmitting;

  const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();

    if (!canSubmit) {
      return;
    }

    const safePrompt = shouldMaskPii ? maskDetectedPii(prompt, detections) : prompt;

    setIsSubmitting(true);
    setErrorMessage(null);

    try {
      await onSubmitPrompt(safePrompt);
      setPrompt('');
    } catch (error) {
      setErrorMessage(error instanceof Error ? error.message : 'Không thể gửi yêu cầu AI.');
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <form className="safe-ai-prompt" onSubmit={handleSubmit}>
      <label htmlFor="safe-ai-prompt-input">Nội dung gửi cho AI</label>
      <textarea
```

```
        id="safe-ai-prompt-input"
        value={prompt}
        rows={6}
        maxLength={5000}
        onChange={(event) => setPrompt(event.target.value)}
      />

      {hasPii && (
        <div className="warning-box" role="alert">
          <strong>Phát hiện dữ liệu cá nhân.</strong>
          <p>Hệ thống khuyến nghị masking trước khi gửi lên AI Cloud.</p>
          <label>
            <input
              type="checkbox"
              checked={shouldMaskPii}
              onChange={(event) => setShouldMaskPii(event.target.checked)}
            />
            Tự động ẩn danh dữ liệu nhạy cảm trước khi gửi
          </label>
        </div>
      )}

      {errorMessage && (
        <p className="form-error" role="alert">
          {errorMessage}
        </p>
      )}

      <button type="submit" className="button button--primary" disabled={!canSubmit}>
        {isSubmitting ? 'Đang gửi...' : 'Gửi cho AI'}
      </button>
    </form>
  );
}
```

5.3.2. Data Classification UI Matrix và hard restriction

Cấp độ	Ví dụ trên UI	Hành vi UI bắt buộc
Public	Câu hỏi chung, mô tả sản phẩm công khai, FAQ	Cho phép gửi bình thường
Internal	Quy trình nội bộ, checklist, tài liệu training chưa public	Cảnh báo nhẹ, yêu cầu xác nhận nếu chính sách nội bộ yêu cầu
Confidential	Email, số điện thoại, CV, ticket khách hàng	Cho phép gửi sau khi masking, hiển thị warning rõ
Restricted	Mật khẩu thô, mã PIN, access token, refresh token, API key, số thẻ tín dụng toàn vẹn	Chặn đứng submit, disable nút gửi, hiển thị cảnh báo đỏ, không cho checkbox masking thông thường

Quy tắc cứng: khi phát hiện RESTRICTED, UI không được để người dùng “tự chịu trách nhiệm rồi gửi”. Đây không còn là UX mềm. Đây là hàng rào bảo mật bắt buộc.

src/features/ai-input/SafeAiPromptBox.tsx

```
import { useMemo, useState } from 'react';
import {
  classifyPromptData,
  maskDetectedPii,
  type DataRiskLevel,
} from '../../shared/privacy/piiDetection';

type SafeAiPromptBoxProps = {
  onSubmitPrompt: (prompt: string) => Promise<void>;
};

function getRiskLabel(riskLevel: DataRiskLevel): string {
  const labels: Record<DataRiskLevel, string> = {
    NONE: 'Chưa có dữ liệu',
    PUBLIC: 'Public',
    INTERNAL: 'Internal',
    CONFIDENTIAL: 'Confidential',
    RESTRICTED: 'Restricted - Cấm gửi',
  };

  return labels[riskLevel];
}

export function SafeAiPromptBox({ onSubmitPrompt }: SafeAiPromptBoxProps) {
  const [prompt, setPrompt] = useState('');
  const [shouldMaskPii, setShouldMaskPii] = useState(true);
  const [isSubmitting, setIsSubmitting] = useState(false);
```

```
const [errorMessage, setErrorMessage] = useState<string | null>(null);

const classification = useMemo(() => classifyPromptData(prompt), [prompt]);
const isRestricted = classification.riskLevel === 'RESTRICTED';
const canSubmit = classification.canSubmit && !isSubmitting && !isRestricted;

const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
  event.preventDefault();

  if (!canSubmit) {
    return;
  }

  const safePrompt = classification.requiresMasking
    ? maskDetectedPii(prompt, classification.detections)
    : prompt;

  setIsSubmitting(true);
  setErrorMessage(null);

  try {
    await onSubmitPrompt(safePrompt);
    setPrompt('');
    setShouldMaskPii(true);
  } catch (error) {
    setErrorMessage(error instanceof Error ? error.message : 'Không thể gửi yêu cầu AI.');
  } finally {
    setIsSubmitting(false);
  }
};

return (
  <form className="safe-ai-prompt" onSubmit={handleSubmit}>
    <div className="form-header">
      <label htmlFor="safe-ai-prompt-input">Nội dung gửi cho AI</label>
      <span className={`risk-badge risk-badge--${classification.riskLevel.toLowerCase()}`}>
        {getRiskLabel(classification.riskLevel)}
      </span>
    </div>

    <textarea
      id="safe-ai-prompt-input"
      value={prompt}
      rows={6}
      maxLength={5000}
      aria-describedby="safe-ai-prompt-help"
      onChange={(event) => setPrompt(event.target.value)}
    />
    <p id="safe-ai-prompt-help" className="text-muted">
      Không nhập mật khẩu, token, mã PIN, API key hoặc số thẻ vào prompt gửi AI.
```

```
      </p>

      {classification.warningMessage && (
        <div className={isRestricted ? 'danger-box' : 'warning-box'} role="alert">
          <strong>{isRestricted ? 'Đã chặn dữ liệu Restricted.' : 'Cần kiểm soát dữ liệu trước khi
gửi.'}</strong>
          <p>{classification.warningMessage}</p>

          {classification.requiresMasking && !isRestricted && (
            <label>
              <input
                type="checkbox"
                checked={shouldMaskPii}
                onChange={(event) => setShouldMaskPii(event.target.checked)}
              />
              Tự động ẩn danh dữ liệu nhạy cảm trước khi gửi
            </label>
          )}
        </div>
      )}

      {classification.detections.length > 0 && (
        <section className="detection-list" aria-labelledby="detected-data-title">
          <h3 id="detected-data-title">Dữ liệu nhạy cảm đã phát hiện</h3>
          <ul>
            {classification.detections.map((item) => (
              <li key={`${item.type}-${item.startIndex}-${item.endIndex}`}>
                <strong>{item.type}</strong> - {item.riskLevel}
              </li>
            ))}
          </ul>
        </section>
      )}

      {errorMessage && (
        <p className="form-error" role="alert">
          {errorMessage}
        </p>
      )}

      <button type="submit" className="button button--primary" disabled={!canSubmit ||
!shouldMaskPii}>
        {isSubmitting ? 'Đang gửi...' : isRestricted ? 'Đã chặn gửi' : 'Gửi cho AI'}
      </button>
    </form>
  );
}
```

5.4. Code mẫu: Accessible AI live region

src/features/ai-accessibility/AiLiveRegion.tsx

```
import { useEffect, useState } from 'react';

type AiLiveRegionProps = {
  message: string;
  politeness?: 'polite' | 'assertive';
};

export function AiLiveRegion({ message, politeness = 'polite' }: AiLiveRegionProps) {
  const [announcedMessage, setAnnouncedMessage] = useState(message);

  useEffect(() => {
    const timeoutId = window.setTimeout(() => {
      setAnnouncedMessage(message);
    }, 250);

    return () => {
      window.clearTimeout(timeoutId);
    };
  }, [message]);

  return (
    <div className="sr-only" role="status" aria-live={politeness} aria-atomic="true">
      {announcedMessage}
    </div>
  );
}
```

src/features/ai-accessibility/AiFallbackNotice.tsx

```
type AiFallbackNoticeProps = {
  title?: string;
  message?: string;
  actionLabel?: string;
  onAction?: () => void;
};

export function AiFallbackNotice({
  title = 'Yêu cầu cần được xử lý thủ công',
  message = 'Hệ thống đã chuyển yêu cầu sang luồng kiểm tra an toàn để đảm bảo độ chính
xác.',
  actionLabel,
  onAction,
}: AiFallbackNoticeProps) {
  return (
    <section className="ai-fallback-notice" role="alert" aria-live="assertive">
```

```
      <h2>{title}</h2>
      <p>{message}</p>
      {actionLabel && onAction && (
        <button type="button" className="button button--secondary" onClick={onAction}>
          {actionLabel}
        </button>
      )}
    </section>
  );
}
```

5.5. Mini-checklist trước khi mở PR

Mục kiểm tra	**Đạt khi**
Live region phù hợp	Cập nhật tiến trình dùng polite, lỗi dùng alert
Không spam screen reader	Không announce từng token nếu stream quá nhanh
PII warning	Email/số điện thoại được phát hiện trước submit
Restricted hard-block	Mật khẩu/PIN/token/số thẻ bị chặn hoàn toàn
Fallback trung tính	Không lộ rule bảo mật hoặc provider nội bộ

6. CHECKLIST TỔNG HỢP & CÁC PHỤ LỤC

Checklist tổng hợp cho dự án ReactJS AI-First OJT 2026

Nhóm	Câu hỏi nghiệm thu
AI-First UX	Người dùng có hiểu AI đang đề xuất hay đang thực thi không?
Human Control	Action nhạy cảm có confirmation không?
Latency	Có progressive loading thay vì spinner vô hạn không?
Error Resilience	AI output sai schema có làm crash toàn app không?
Streaming	Có stop/cancel và cleanup stream không?
Security UI	AI output có được sanitize trước khi render HTML không?
Privacy	Có phát hiện, masking hoặc hard-block dữ liệu nhạy cảm không?
A11Y	Screen reader có nhận được trạng thái quan trọng không?

Nhóm	Câu hỏi nghiệm thu
PR Evidence	PR có screenshot, video hoặc test evidence cho fallback/negative case không?

Phụ lục A: Folder structure gợi ý

```
src/
  app/
    App.tsx
    main.tsx
    routes.tsx
  features/
    ai-actions/
      types.ts
      AiSuggestedActionCard.tsx
      ConfirmActionDialog.tsx
      useOptimisticAiAction.ts
    ai-progress/
      aiProgressTypes.ts
      AiProgressTimeline.tsx
    ai-result/
      AiResultSchemaGuard.tsx
      AiSummaryPanel.tsx
    ai-streaming/
      aiTaskTypes.ts
      useAiTaskEvents.ts
      AiTaskStatusPanel.tsx
      useTextStream.ts
      useBufferedTextStream.ts
      useEphemeralAiBuffer.ts
      StreamingAnswerBox.tsx
      AiStreamingDemo.tsx
    ai-input/
      SafeAiPromptBox.tsx
    ai-accessibility/
      AiLiveRegion.tsx
      AiFallbackNotice.tsx
  shared/
    components/
      SafeAiText.tsx
    error/
      AiErrorBoundary.tsx
    privacy/
      piiDetection.ts
    utils/
      sanitizeHtml.ts
  styles/
    ai-demo.css
```

Phụ lục B: Definition of Done cho ReactJS AI-First

Mã DoD	Tiêu chí
DOD-FE-AI-01	Giao diện phân biệt rõ AI Suggestion và User-confirmed Action
DOD-FE-AI-02	Mọi hành động rủi ro có confirmation dialog và trạng thái disabled khi đang xử lý
DOD-FE-AI-03	Loading state thể hiện pipeline thật, không dùng spinner vô hạn đơn độc
DOD-FE-AI-04	Component hiển thị AI output được bọc Error Boundary hoặc Schema Guard
DOD-FE-AI-05	SSE/EventSource và Fetch stream có cleanup khi unmount
DOD-FE-AI-06	Người dùng có thể abort stream đang chạy
DOD-FE-AI-07	PII được cảnh báo/masking, Restricted data bị hard-block
DOD-FE-AI-08	UI đạt A11Y tối thiểu với keyboard flow, aria-live, role=status/alert

Phụ lục C: 20 câu hỏi Mock Interview Frontend AI-First

1. Vì sao Frontend được gọi là AI Experience Plane?
2. Khác biệt giữa AI Suggestion và User-confirmed Action là gì?
3. Khi nào bắt buộc dùng confirmation dialog cho AI action?
4. Optimistic UI trong sản phẩm AI khác gì optimistic UI của CRUD?
5. Nếu AI output sai schema, Frontend phải xử lý như thế nào?
6. Error Boundary bắt được loại lỗi nào và không bắt được loại lỗi nào?
7. Vì sao không nên render AI output bằng dangerouslySetInnerHTML?
8. Bạn sẽ sanitize output AI như thế nào để giảm rủi ro XSS?
9. SSE phù hợp với bài toán nào trong AI UI?
10. Fetch Streaming khác SSE ở điểm nào?
11. AbortController giúp tiết kiệm chi phí token như thế nào?
12. Vì sao không nên setState sau từng token?
13. Khi nào nên dùng useRef buffer cho streaming text?
14. Vì sao không được lưu prompt nhạy cảm vào localStorage?
15. Dữ liệu nào phải bị hard-block trước khi gửi lên AI Cloud?
16. aria-live="polite" và role="alert" khác nhau thế nào?
17. Làm sao tránh spam screen reader khi AI stream liên tục?
18. Fallback message trung tính nên viết như thế nào?
19. Bạn sẽ viết PR evidence cho tính năng AI streaming ra sao?
20. Một Frontend AI-First đạt chuẩn OJT 2026 cần vượt qua những negative test nào?

Đừng chỉ đọc code suông. 20 câu hỏi trên chính là bộ khung mà các doanh nghiệp công nghệ lớn sẽ dùng để chất vấn em khi phỏng vấn vị trí Frontend AI-First năm 2026.

ĐĂNG KÝ NGAY 1 SUẤT MOCK INTERVIEW 1-1 MIỄN PHÍ 15 PHÚT VỚI MENTOR TRƯỞNG để thử khả năng phòng thủ, giải trình logic dòng dữ liệu và rà soát A11Y/PII của mình.

Em sẽ nhận feedback thực tế theo Rubric doanh nghiệp và một lộ trình cá nhân hóa gửi thẳng vào email sau buổi phỏng vấn. Link đăng ký: https://ojt.r2s.vn/

Giới hạn tối đa 15 suất đăng ký thành công trong tuần này để đảm bảo chất lượng tương tác tốt nhất.

Phụ lục D: CSS tối thiểu cho demo OJT

```css
:root {
  font-family: Inter, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: #0f172a;
  background: #f8fafc;
}

.button {
  border: 0;
  border-radius: 10px;
  cursor: pointer;
  font-weight: 700;
  padding: 10px 16px;
}

.button:disabled {
  cursor: not-allowed;
  opacity: 0.55;
}

.button--primary {
  background: #2563eb;
  color: #ffffff;
}

.button--secondary {
  background: #e2e8f0;
  color: #0f172a;
}

.button--danger {
  background: #dc2626;
  color: #ffffff;
}

.button--ghost {
```

```css
  background: transparent;
  color: #2563eb;
}

.ai-action-card,
.ai-summary-panel,
.streaming-answer,
.safe-ai-prompt,
.ai-fallback-notice {
  background: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 18px;
  box-shadow: 0 12px 32px rgba(15, 23, 42, 0.08);
  margin: 16px 0;
  padding: 20px;
}

.risk-badge {
  border-radius: 999px;
  display: inline-flex;
  font-size: 12px;
  font-weight: 800;
  padding: 4px 10px;
  text-transform: uppercase;
}

.risk-badge--low,
.risk-badge--public {
  background: #dcfce7;
  color: #166534;
}

.risk-badge--medium,
.risk-badge--confidential {
  background: #fef9c3;
  color: #854d0e;
}

.risk-badge--high,
.risk-badge--restricted {
  background: #fee2e2;
  color: #991b1b;
}

.warning-box {
  background: #fffbeb;
  border: 1px solid #f59e0b;
  border-radius: 12px;
  color: #78350f;
  margin: 12px 0;
```

```
  padding: 12px;
}

.danger-box,
.form-error {
  background: #fef2f2;
  border: 1px solid #ef4444;
  border-radius: 12px;
  color: #991b1b;
  margin: 12px 0;
  padding: 12px;
}

.sr-only {
  border: 0;
  clip: rect(0 0 0 0);
  height: 1px;
  margin: -1px;
  overflow: hidden;
  padding: 0;
  position: absolute;
  white-space: nowrap;
  width: 1px;
}
```

Phụ lục E: Negative test cases bắt buộc cho Frontend AI-First

ID	Input	Expected Behavior
FE-NEG-01	AI output chứa <script>alert(1)</script>	Script bị loại bỏ, UI không thực thi mã
FE-NEG-02	AI output chứa <img src=x onerror=alert(1)>	Thuộc tính onerror bị loại bỏ
FE-NEG-03	Prompt chứa password=abc123456	Nút gửi bị disabled, cảnh báo Restricted xuất hiện
FE-NEG-04	Prompt chứa số thẻ hợp lệ	UI hard-block submit
FE-NEG-05	Stream chạy dài, user bấm dừng	AbortController hủy request, UI chuyển ABORTED
FE-NEG-06	SSE mất kết nối	UI hiển thị trạng thái disconnected và không crash
FE-NEG-07	AI result thiếu confidenceScore	Schema guard throw error, Error Boundary hiển thị fallback

ID	Input	Expected Behavior
FE-NEG-08	Component unmount khi stream đang chạy	Cleanup event source, buffer và timer

Nguồn kỹ thuật tham khảo

- React 19 official blog: Error handling improvements and React 19 release notes.
- React docs: Error Boundaries and component error handling.
- MDN Web Docs: Server-Sent Events, EventSource, AbortController.
- Vite official guide: React TypeScript template and modern frontend tooling.